12장 고급 기능

깃은 안전한 저장소를 유지하는 고급 명령어들을 제공합니다. 이 장에서는 각 장에서 다루지 못했던 몇 가지 고급 기능을 모아서 알아보겠습니다.

12.1 refs

12.2 reflog

12.3 파일 애너테이션

12.4 replace

12.5 가비지 콜렉트

12.6 prune

12.7 rerere

12.8 정리

12.1 refs

깃은 커밋으로 코드 이력을 관리합니다. 커밋은 고유의 SHA1 해시 값을 가지고 있으며, 이 해시 값은 여러 기능에서 참조합니다. 깃에서는 참조하는 해시 값을 refs 목록으로 가지고 있습니다.

12.1.1 실습 환경 준비

먼저 refs 기능을 학습할 수 있도록 실습 환경을 만들겠습니다. 저장소를 생성합니다.

```
$ cd 실습폴더
```

```
$ mkdir gitstudy12 새 폴더 만들기
```

```
$ cd gitstudy12
```

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)
```

```
$ git init 깃 초기화
```

```
Initialized empty Git repository in E:/gitstudy12/.git/
```

index.htm 파일을 생성합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)
```

```
$ code index.htm VS Code 실행
```

index.htm

```
hello world
```

간단한 인사말만 작성했습니다. 저장한 후 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git add index.htm 등록




infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git commit -m "first" 커밋

[main (root-commit) d0943cf] first

 1 file changed, 1 insertion(+)

 create mode 100644 index.htm
```

12.1.2 해시

깃에서 해시 값은 매우 중요합니다. 깃은 SHA1 알고리즘을 사용하여 해시 값을 생성합니다. 해시 값은 깃의 동작을 구분하며, 중복되지 않는 유일한 값입니다.

깃의 모든 작업은 SHA1 해시 값을 참조합니다. 깃 내부적으로 동작하는 작업들은 SHA1 해시 값으로 연결 고리를 생성합니다. 따라서 깃의 동작을 정확히 이해하려면 해시 값을 자세히 알아볼 필요가 있습니다.

생성된 모든 해시 값은 show 명령어로 확인할 수 있습니다.

$ git show 해시값

저장소의 로그를 확인해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git log 커밋 로그


commit d0943cfbc5e092668be3b96e98f32e363e05feb1 (HEAD → main)


Author: hojin <infohojin@gmail.com>


Date:   Sat May 25 18:06:47 2019 +0900


    first
```

커밋 로그가 1개 출력됩니다. 출력된 d0943cf 해시 값의 정보를 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git show d0943cf 커밋 정보

commit d0943cfbc5e092668be3b96e98f32e363e05feb1 (HEAD → main)

Author: hojin <infohojin@gmail.com>

Date:   Sat May 25 18:06:47 2019 +0900

    first

diff --git a/index.htm b/index.htm

new file mode 100644

index 0000000..95d09f2

--- /dev/null

+++ b/index.htm

@@ -0,0 +1 @@
```

```
+hello world


\ No newline at end of file
```

해시 값을 사용하여 커밋의 상세 정보를 확인할 수 있습니다.

12.1.3 역조회

show 명령어는 해시 값을 사용하여 커밋 정보를 확인합니다. 반대로 rev-parse 명령어로 포인터의 해시 값을 알 수 있습니다. 예를 들어 브랜치는 커밋 해시 값을 가리키는 포인터입니다. 따라서 브랜치 이름을 사용하여 참조하는 해시 값을 조회할 수 있습니다. main 브랜치의 해시 값을 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git rev-parse main


d0943cfbc5e092668be3b96e98f32e363e05feb1
```

12.1.4 참조 목록

깃은 SHA1 해시 값을 생성하고, 커밋은 생성된 해시 값을 간접적으로 사용합니다. 또 깃에서는 생성된 해시 값을 쉽게 참조할 수 있도록 refs 목록을 생성합니다. 깃의 모든 refs 목록은 저장소의 숨긴 영역인 .git/refs 폴더 안에 저장됩니다.

또 복잡한 SHA1 해시 값을 쉽게 찾아 사용할 수 있도록 별칭도 쓸 수 있습니다. 별칭은 .git/refs 폴더 안에서 생성 및 관리할 수 있습니다. 즉, refs 정보는 깃의 기능들을 구현하는 내부 메커니즘입니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ ls .git/refs -all 저장소 refs 파일 목록


total 4


drwxr-xr-x 1 infoh 197609 0 5월  25 18:02 .


drwxr-xr-x 1 infoh 197609 0 5월  25 18:06 ..


drwxr-xr-x 1 infoh 197609 0 5월  25 18:06 heads


drwxr-xr-x 1 infoh 197609 0 5월  25 18:02 tags
```

처음 저장소를 생성하면 .git/refs 폴더에는 heads와 tags 폴더만 있습니다. 새로운 브랜치를 만들 때마다 해시 값을 가지는 refs 파일들을 생성합니다.

새로운 feature 브랜치를 만들고 .git/refs 폴더를 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git branch feature 브랜치 생성



infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ ls .git/refs/heads -all 저장소 refs 정보

total 2

drwxr-xr-x 1 infoh 197609  0 5월  25 18:14 .

drwxr-xr-x 1 infoh 197609  0 5월  25 18:02 ..

-rw-r--r-- 1 infoh 197609 41 5월  25 18:14 feature 브랜치의 HEAD 포인트

-rw-r--r-- 1 infoh 197609 41 5월  25 18:06 main
```

feature 브랜치의 refs가 생성되었습니다.

12.2 reflog

깃은 안정적인 작업을 유지하려고 참조된 모든 refs를 기록합니다. 그리고 내부적으로 작업한 모든 HEAD와 브랜치 포인터를 기록합니다. 이때 사용된 포인터들의 기록을 reflog라고 합니다.

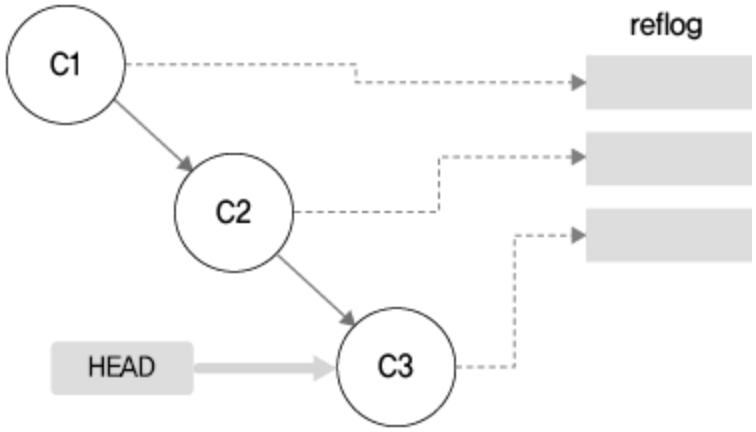
12.2.1 참조 기록

reflog 기록은 reflog 명령어를 사용하여 확인할 수 있습니다.

$ git reflog

reflog는 한 번이라도 사용했던 HEAD와 브랜치를 기록합니다. 하지만 모든 기록을 영구적으로 가지고 있지는 않습니다. reflog는 시스템에서 정의한 며칠 간의 기록만 보관합니다. 그 이전의 작업들은 모두 삭제합니다.

▼ 그림 12-1 ref 기록



지금까지 작업한 reflog 기록을 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git reflog reflog 기록


d0943cf (HEAD → main, feature) HEAD@{0}: commit (initial): first


...
```

12.2.2 기록 확인

reflog의 기록들은 HEAD@{숫자} 형태입니다. 각 숫자는 작업을 수행한 해시 값을 가리킵니다. 따라서 reflog에 기록된 HEAD@{숫자} 포인터를 이용하여 커밋 정보를 확인할 수 있습니다.

마지막에 참조한 refs 로그의 해시 값 정보를 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git show HEAD@{0} 커밋 정보

commit d0943cfbc5e092668be3b96e98f32e363e05feb1 (HEAD → main, feature)

Author: hojin <infohojin@gmail.com>

Date:   Sat May 25 18:06:47 2019 +0900

    first




diff --git a/index.htm b/index.htm

new file mode 100644

index 0000000..95d09f2

--- /dev/null

+++ b/index.htm

@@ -0,0 +1 @@

+hello world
```

\ No newline at end of file

12.2.3 기간 확인

커밋의 로그 기록이 많은 경우 필터링할 수 있습니다. 필터링은 특정한 날짜, 시간 등 기준을 적용합니다. 다음 예처럼 어제 작업한 내역만 출력할 수 있습니다.

예

```
$ git show main@{yesterday}
```

깃에서 참조하는 refs의 로그는 reflog 명령어 또는 git log -g 옵션을 사용하여 확인할 수 있습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git log -g 커밋 로그


commit d0943cfbc5e092668be3b96e98f32e363e05feb1 (HEAD → main, feature)


Reflog: HEAD@{0} (hojin <infohojin@gmail.com>)


Reflog message: commit (initial): first


Author: hojin <infohojin@gmail.com>


Date:   Sat May 25 18:06:47 2019 +0900


    first


...
```

12.2.4 기록 유지

refs는 현재 로컬 저장소에서 작업한 모든 로그의 참조 기록입니다. 깃은 커밋, 브랜치 등 내부적으로 사용한 모든 객체의 로그를 기록합니다. 하지만 refs는 로컬 저장소에만 기록하므로 원격 저장소나 복제, 복사 등으로는 refs 기록을 옮길 수 없습니다.

12.3 파일 애너테이션

개발하다 보면 코드를 잘못 작성해서 오류가 발생하곤 합니다. 하지만 여러 개발자와 협업하다 보면 잘못된 코드를 찾는 것이 쉽지 않습니다.

12.3.1 blame

잘못된 코드가 어디서부터 시작되었는지 찾기 어렵습니다. 잘못된 내용을 찾으려면 모든 커밋 이력을 살펴보아야 하는데, 생각보다 시간이 오래 걸립니다. 깃은 이러한 코드를 쉽게 찾을 수 있게 파일의 수정 이력을 분석합니다. 그리고 blame 기능은 커밋의 메타 정보를 코드 라인별로 같이 결합하여 출력합니다. 코드를 수정한 사람이 누구인지, 언제 수정한지를 쉽게 판별할 수 있으며, 메타 정보를 바탕으로 문제를 좀 더 쉽게 파악할 수 있습니다.

12.3.2 실습 환경 준비

blame 기능을 실습하기 위해 index.htm 파일을 수정한 후 커밋해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ code index.htm VS Code 실행

index.htm

<h1>hello world</h1>

infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git commit -am "add h1 tag" 등록 및 커밋


[main 520aadf] add h1 tag


 1 file changed, 1 insertion(+), 1 deletion(-)




infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ code index.htm VS Code 실행

index.htm

<h1>hello world</h1>


깃을 이용하여 코드 이력을 관리할 수 있습니다.
```

infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git commit -am "add description" 등록 및 커밋

[main 023dade] add description

 1 file changed, 2 insertions(+), 1 deletion(-)

infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ code index.htm VS Code 실행

index.htm

<h1>hello world</h1>

깃을 이용하여 코드 이력을 관리할 수 있습니다.

깃은 ref를 참조하여 작업이 이루어집니다.

infoh@DESKTOP MINGW64 /e/gitstudy12 (main)

$ git commit -am "ref description" 등록 및 커밋

[main 71099d0] ref description

 1 file changed, 2 insertions(+), 1 deletion(-)

12.3.3 blame 명령어

blame 명령어는 개별 파일에서만 동작하며, 명령어 인자 값으로 개별 파일을 전달합니다.

$ git blame 파일이름

작성한 파일의 blame을 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git blame index.htm 메타 정보 출력


023dadec (hojin 2019-05-26 16:19:05 +0900 1) <h1>hello world</h1>


71099d05 (hojin 2019-05-26 16:31:18 +0900 2) 깃을 이용하여 코드 이력을 관리할 수 있습니다.


71099d05 (hojin 2019-05-26 16:31:18 +0900 3) 깃은 ref를 참조하여 작업이 이루어집니다.
```

출력 결과를 보니 각 커밋에 대한 해시 값, 작성자, 코드 등 내용도 함께 보여 주네요. blame은 누가 코드의 어느 라인을 수정했는지 파악할 때 유용합니다.

12.3.4 옵션 활용

소스 코드의 용량이 클 때는 이력 정보도 많이 출력됩니다. 이때는 -L 옵션을 사용하여 파일의 특정 영역만 지정할 수 있습니다.

$ git blame -L 시작줄, 마지막줄 파일이름

파일을 수정한 히스토리를 출력해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy12 (main)


$ git blame -L 2,3 index.htm 메타 정보 필터링


71099d05 (hojin 2019-05-26 16:31:18 +0900 2) 깃을 이용하여 코드 이력을 관리할 수 있습니다.


71099d05 (hojin 2019-05-26 16:31:18 +0900 3) 깃은 ref를 참조하여 작업이 이루어집니다.
```

blame 기능은 파일에서 특정한 수정 사항 및 커밋들을 찾는 데 매우 유용합니다. 개발 과정에서 만든 수많은 코드 중 수정한 부분만 쉽게 찾아낼 수 있습니다.

blame 명령어는 옵션을 사용하면 좀 더 다양하게 검색할 수 있습니다.

- -e: 사용자 이름 대신 이메일을 출력합니다.
- -w: 공백 문자를 무시합니다.
- -M: 같은 파일 내에서 복사나 이동을 감지합니다.
- -C: 다른 파일에서 이동이나 복사된 것을 감지할 수 있습니다.

12.4 replace

기본적으로 한 번 생성된 객체는 변경할 수 없습니다. 커밋을 변경하려면 깃의 원리를 응용해야 합니다. 커밋이 참조하는 해시 값을 속여서 다른 커밋으로 변경하는 것입니다. replace는 기존 커밋을 다른 커밋인 것처럼 변경하는 기능입니다.

깃 저장소에서 오랫동안 작업하면 수많은 커밋 기록이 쌓입니다. 커밋이 많아지면 저장소 크기도 증가합니다. 저장소를 분리할 수 있는 서브모듈이 있지만, 오래된 저장소와는 다릅니다. 이때 replace를 사용하면 커밋이 많은 오래된 저장소를 분리할 수 있습니다.

12.4.1 실습 환경 준비

replace 명령어를 사용하여 저장소를 분리해 봅시다. 실습을 위해 새로운 저장소를 생성합니다.

```
$ cd 실습폴더


$ mkdir git-replace 새 폴더 만들기


$ cd git-replace
```

```
infoh@DESKTOP MINGW64 /e/git-replace (main)


$ git init 저장소 초기화


Initialized empty Git repository in E:/git-replace/.git/
```

실습을 위해 work.md 파일을 생성하고 저장한 후 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/git-replace (main)


$ code work.md VS Code 실행
```

work.md

```
# 작업1
```

```
infoh@DESKTOP MINGW64 /e/git-replace (main)

$ git add work.md 등록




infoh@DESKTOP MINGW64 /e/git-replace (main)

$ git commit -m "work1" 커밋

[main (root-commit) 6584676] work1

 1 file changed, 1 insertion(+)

 create mode 100644 work.md
```

12.4.2 저장소 분리

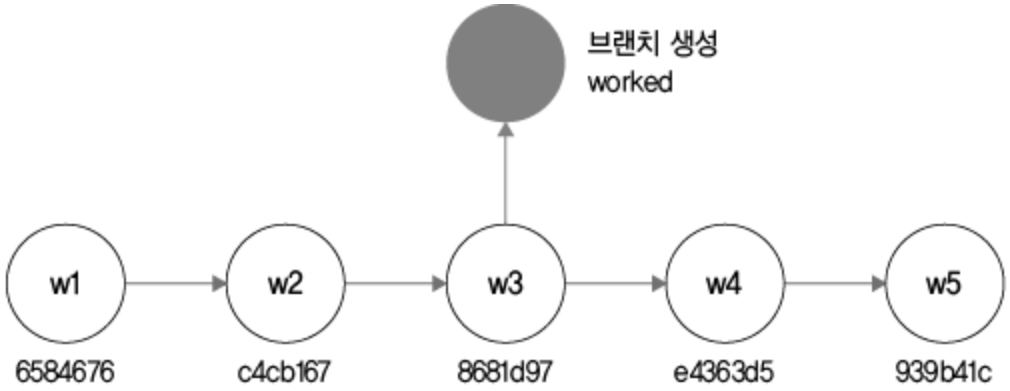
준비한 저장소에는 커밋이 4개 있습니다. 이 저장소를 2개로 분리해 보겠습니다.

1단계. 저장소를 분리할 수 있게 새로운 브랜치를 하나 생성합니다. 작업3을 기준으로 브랜치를 만듭니다.

```
infoh@DESKTOP MINGW64 /e/git-replace (main)
```

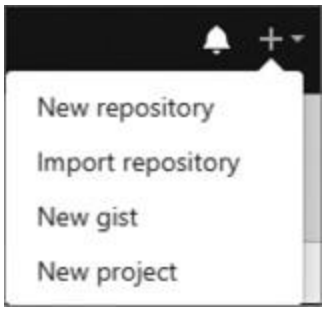
```
$ git branch worked 8681d97 work3 해시 값 입력
```

▼ 그림 12-3 분리할 위치에 브랜치 생성



깃허브에서 git-replace 저장소와 동기화할 새로운 원격 저장소를 생성합니다. 먼저 깃허브에 로그인한 후 New 버튼 또는 + > New repository를 클릭합니다.

▼ 그림 12-4 깃허브에서 새 저장소 메뉴 선택



12.4.3 저장소 분리

앞에서 우리는 저장소를 분리하려고 커밋을 지정했습니다. 또 지정한 커밋을 기준으로 브랜치를 만들고, 이를 원격 저장소의 다른 브랜치 이름으로 전송했습니다. 하지만 아직 로컬 저장소는 변경된 것이 없습니다. 로컬 저장소는 모든 커밋 정보를 가지고 있습니다. 이제 실제적인 저장소를 분리해 보겠습니다.

4단계. 분리를 하려면 가상의 임시 객체를 하나 생성해야 합니다. 이때는 commit-tree 명령어를 사용합니다. commit-tree는 저수준 깃 명령어입니다.

commit-tree 명령어를 사용하여 수동으로 객체를 생성합니다.

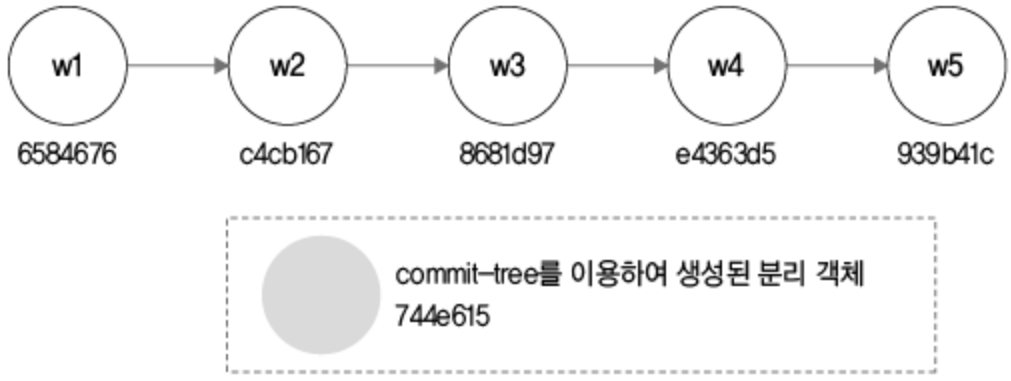
```
infoh@DESKTOP MINGW64 /e/git-replace (main)


$ echo "devide history" | git commit-tree c4cb167^{tree} work2의 해시 값


744e61522896c3ee222fd4f74a099964851454cf
```

저수준 명령어를 사용하여 임시 객체를 생성했습니다. 임시 객체는 아직 다른 어떤 객체와도 연결되지 않고, 떨어져 있는 외톨이 객체입니다.

▼ 그림 12-8 임시 객체 생성



12.4.4 저장소 연결

임시 객체를 이용하여 로컬 저장소의 커밋 연결 고리를 분리했습니다. 연결 고리가 분리된 커밋들은 각각 두 원격 저장소로 푸시되었습니다. 이번에는 분리된 두 저장소를 저장소 하나로 연결해 보겠습니다.

7단계. 분리된 두 원격 저장소 중에서 마지막에 작업한 커밋으로 원격 저장소를 복제합니다.

```
$ cd 실습폴더


$ git clone https://github.com/jinygit/git-replace2.git git-replace2 저장소 복제


Cloning into 'git-replace2'...


remote: Enumerating objects: 12, done.


remote: Counting objects: 100% (12/12), done.


remote: Compressing objects: 100% (5/5), done.


remote: Total 12 (delta 1), reused 12 (delta 1), pack-reused 0


Unpacking objects: 100% (12/12), done.
```

복제된 로컬 저장소의 로그를 확인합니다.

```
infoh@DESKTOP MINGW64 (main)

$ cd git-replace2 복제 저장소 이동



infoh@DESKTOP MINGW64 /e/git-replace2 (main)

$ git log --oneline --decorate 커밋 기록

c282ad0 (HEAD → main, origin/main, origin/HEAD) work5

461d0a2 work4

c3502a6 work3

744e615 devide history
```

12.5 가비지 콜렉트

깃은 저장소를 효율적으로 유지 관리하려고 가비지 콜렉트(garbage collect)를 지원합니다. 또 깃은 가비지를 효과적으로 관리할 수 있는 별도의 명령어를 제공합니다.

12.5.1 가비지

가비지(garbage)는 동적인 언어에서 메모리를 관리하려고 생성한 개념입니다. 깃은 이력을 추적할때 객체의 생성과 변경을 반복합니다. 여러 번 처리 동작을 반복하면서 시간이 지남에 따라 연결 고리가 없는 고립된 객체들이 생겨 비효율적인 자원 상태가 됩니다.

고립된 객체들은 대표적으로 리셋(reset) 또는 리베이스(rebase) 등을 자주 할 때 발생합니다. 연결 고리가 없는 객체들은 불필요하며, 용량만 차지하므로 정리해 주는 작업이 필요합니다.

Note

깃의 커밋은 생성된 객체의 연결 고리를 설정하는 동작입니다. 커밋을 변경하면 새로운 객체로 연결 고리를 재설정합니다. 하지만 기존에 연결된 객체들은 삭제하지 않고 유지합니다. 이러한 과정에서 불필요한 가비지가 생성됩니다. 이처럼 객체는 향후 수동으로 커밋을 복구할 때 사용할 수 있기 때문에 유지합니다.

12.5.2 압축 관리

깃의 내부 원리는 SHA1 해시와 객체의 응용입니다. 파일 변경, 트리 구조, 커밋 등 대부분의 내부 작업은 객체를 생성하고 연결하는 동작들입니다. 커밋들이 실행될 때 이러한 내부 동작으로 연결 고리가 없는 객체가 수없이 생성됩니다. 깃 내부에 이러한 객체가 많아지면 저장소 용량도 커지고, 객체도 빠르게 관리하기 어렵습니다. 깃은 이러한 객체를 줄이려고 생성된 객체를 압축합니다. 즉, 깃은 연결 고리가 없는 객체들을 pack 파일 형태로 압축하여 저장합니다.

12.5.3 실행

깃은 가비지를 정리하려고 별도의 gc 명령어를 제공합니다. gc는 저수준 명령어로 garbage collect의 약어입니다. 깃에서 내부적으로 가비지 정리가 필요하다고 생각할 때, gc 명령어를 자동으로 실행합니다. gc 명령어가 실행되면 오래된 객체들은 삭제하고 저장소 용량도 정리합니다.

자동 실행 외에 사용자가 직접 gc 명령어를 실행할 수 있습니다.

$ git gc --auto

gc 명령어를 실행할 때는 prune, repack, pack, rerere 등 하위 명령어와 같이 사용합니다. gc 명령어가 실행되면 객체를 압축하고 pack 파일 형태로 저장하거나 제거합니다.

12.5.4 refs 압축

gc 명령어는 객체의 압축과 refs를 같이 처리합니다. refs를 같이 압축하면 압축하기 전의 파일들은 삭제됩니다. 그리고 새로운 .git/packed-refs 파일을 생성합니다.

이후 refs가 추가로 변경되면 압축한 packed-refs 값을 수정하지 않고 새로운 refs 파일을 생성합니다. 즉, 압축한 이후에는 refs 파일이 2개가 되며, refs 파일이 여러 개 있으면 기본적으로 refs 안에 있는 파일을 먼저 찾습니다. 그리고 이후에 압축된 packed-refs 내용을 검색합니다.

12.5.5 환경 설정

gc 명령어의 동작은 환경 설정으로 제어할 수 있습니다.

- gc.reflogExpire: reflog가 보존되는 기간을 설정합니다. 기본값은 90일입니다.
- gc.reflogExpireUnreachable: 기본값은 30일입니다.
- gc.aggressiveWindow: 창의 크기를 정합니다. 기본값은 250입니다.
- gc.aggressiveDepth: 압축에 사용되는 매개변수이며, 기본값은 50입니다.
- gc.pruneExpire: 저장소에 쓰는 다른 프로세스와 동시에 실행될 때 손상을 방지합니다.
- gc.worktreePruneExpire: 유예 기간을 설정할 수 있습니다.

gc 명령어의 동작은 gc.auto 항목으로 자동 설정을 허용하지 않을 수도 있고, gc.autopacklimit를 사용하여 최대 압축 숫자를 제어할 수도 있습니다.

12.6 prune

깃의 가비지를 정리하는 gc 명령어는 몇 가지 명령어와 함께 사용합니다. 그중 prune 명령어는 고립된 객체를 정리하는 내부 유틸리티입니다.

12.6.1 고립된 객체

깃은 객체를 이용하여 이력을 추적하고 변경된 내역을 기록합니다. 새로운 변경과 커밋이 있을 때 기존 객체를 다른 객체로 연결합니다. 즉, 앞에서도 이야기했듯이 객체들은 SHA1 해시 값을 사용하여 서로 연결되어 있습니다. 어떤 객체를 새로운 커밋으로 재설정하면 참조 링크가 해제되는데, 이것은 해당 객체가 더 이상 필요하지 않다는 것입니다. 즉, 불필요한 객체가 생기는 것입니다.

고립된 객체는 객체 간 연결 고리가 끊겨 명령어를 사용해도 해당 객체에 접근할 수 없는 객체를 의미합니다. 고립된 객체는 불필요하므로 정리해야 합니다. 깃 자체적으로 완벽하게 정리하기 어려우므로 수동으로 정리해야 합니다. 실습으로 이 내용을 익혀 봅시다.

12.6.2 실습 환경 준비

prune 명령어를 실습할 수 있게 저장소를 하나 생성합니다.

```
$ cd 실습폴더


$ mkdir git-prune 새 폴더 만들기


$ cd git-prune
```

```
infoh@DESKTOP MINGW64 /e/git-prune (main)


$ git init 저장소 초기화


Initialized empty Git repository in E:/git-prune/.git/
```

prune.md 파일을 만들고 첫 번째 커밋을 합니다.

```
infoh@DESKTOP MINGW64 /e/git-prune (main)


$ code prune.md VS Code 실행
```

prune.md

```
# Prune 실습입니다.
```

```
infoh@DESKTOP MINGW64 /e/git-prune (main)

$ git add prune.md 등록




infoh@DESKTOP MINGW64 /e/git-prune (main)

$ git commit -m "first" 커밋

[main (root-commit) ea57d7f] first

 1 file changed, 1 insertion(+)

 create mode 100644 prune.md
```

12.6.3 객체 삭제

prune 명령어는 고립된 객체를 정리하는 기능입니다. 따라서 깃 내부에 고립된 객체가 있어야 합니다. 리베이스 또는 리셋 등 명령어 작업들은 고립된 객체를 생성합니다. 실습을 위해 인위적으로 고립된 객체를 생성해 보겠습니다.

먼저 log 명령어로 커밋 기록을 확인합니다.

```
infoh@DESKTOP MINGW64 /e/git-prune (main)


$ git log 커밋 기록


commit 72c9f62bc5f9577250fcc8437a2ad3883e35dff1 (HEAD → main)


Author: hojin <infohojin@gmail.com>


Date:   Tue Feb 19 16:28:38 2019 +0900


    second




commit ea57d7fdd9be86740511c9ca81aba0c004302d5d


Author: hojin <infohojin@gmail.com>


Date:   Tue Feb 19 16:25:43 2019 +0900


    first
```

reset 명령어를 사용하여 마지막 커밋을 강제로 제거합니다.

infoh@DESKTOP MINGW64 /e/git-prune (main)

$ git reset --hard ea57d7f 하드 리셋으로 삭제

HEAD is now at ea57d7f first

12.6.4 객체 정리

리셋으로 제거된 객체는 삭제되지 않고 고립된 객체가 됩니다. 리셋으로 객체까지 완벽하게 삭제하려면 prune 명령어를 사용합니다.

prune 명령어를 실행할 때는 다음 두 옵션을 같이 사용합니다.

- --dry-run: 실행하지 않고 작업할 내역만 출력합니다.
- --verbose: 작업한 결과를 출력합니다.

prune 명령어를 사용하여 남아 있는 고립된 객체를 삭제해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/git-prune (main)


$ git prune --dry-run --verbose 객체 삭제
```

prune 명령어는 실행 후의 작업 결과를 출력합니다. 하지만 어떤 메시지도 출력하지 않았습니다. 이처럼 작업 메시지가 없는 것은 아직 깃 내부에 고립된 객체가 없어 prune 동작 자체가 실행되지 않은 것입니다.

그럼 이전에 리셋으로 제거한 객체는 무엇일까요? 리셋은 단지 커밋의 참고 연결 고리만 해제한 것입니다. 사실 깃은 생성된 객체를 쉽게 삭제하는 것을 허용하지 않습니다. 정확하게 말하자면 참고 연결 고리만 해제한 객체와 실제 고립된 객체는 서로 다릅니다.

12.6.5 원격 작업

보통 prune 명령어는 로컬 환경에서 실행합니다. 하지만 고립된 객체는 원격 저장소에도 있을 수 있습니다. remote/fetch 명령어를 같이 조합하면 원격 저장소에서도 prune 명령어를 실행할 수 있습니다.

$ git remote prune

원격 저장소의 브랜치를 병합한 후 삭제합니다. 삭제된 원격 저장소 브랜치는 더 이상 참조할 수 없습니다.

$ git fetch --prune

fetch --prune 명령어는 오래된 브랜치를 정리합니다. 원격 저장소에 연결한 후 제거하기 전에 최신 상태를 먼저 가져옵니다.

12.7 rerere

불특정 다수의 사람과 협업하다 보면 여러 가지 문제가 발생합니다. 깃은 협업 작업을 할 때 발생한 문제들을 충돌이라는 형태로 알려 줍니다.

12.7.1 동일한 충돌

협업 중 발생하는 여러 문제 중에서 일부는 같은 유형의 문제입니다. 비슷한 충돌 패턴을 반복적으로 수동으로 해결하곤 합니다. 깃은 이렇게 반복되는 같은 유형의 충돌 문제를 자동으로 해결할 수 없을까 하는 의문점에서 등장한 기능입니다.

rerere는 reuse recorded resolution의 약어로, 어떤 문제로 충돌이 발생할 때 이를 기록합니다. rerere 기능을 활성화하면, 깃은 충돌을 해결할 때마다 해결한 문제의 유형을 기록합니다. 기록한 유형의 문제와 비슷한 문제가 향후 다시 발생한다면, 미리 기록한 해결 정보를 바탕으로 자동으로 적용합니다.

12.7.2 활성화

기본적으로 rerere 기능은 활성화되어 있지 않습니다. 이 기능을 사용하려면 다음 명령어로 활성화해야 합니다.

$ git config rerere.enabled true

로컬의 전체 저장소에 모두 적용하고자 한다면 --global 옵션을 같이 사용합니다. 단일 저장소에서만 rerere 기능을 활성화할 때는 글로벌 옵션을 제외합니다. 환경 설정으로 활성화하면 저장소에서 rerere 기능을 사용할 수 있습니다.

12.7.3 실습 준비

rerere 기능을 활성화해 봅시다. 실습을 위해 저장소를 하나 생성합니다.

```
$ cd 실습폴더


$ mkdir git-rerere 새 폴더 만들기


$ cd git-rerere
```

```
infoh@DESKTOP MINGW64 /e/git-rerere (main)


$ git init 저장소 초기화


Initialized empty Git repository in E:/git-rerere/.git/
```

환경 설정에서 rerere 기능을 활성화합니다.

```
infoh@DESKTOP MINGW64 /e/git-rerere (main)


$ git config rerere.enabled true rerere 활성화
```

전체 저장소가 아닌 현재의 로컬 저장소에서만 rerere 기능을 활성화했습니다. 설정된 정보는 .git/config에 저장합니다. 설정 정보를 확인해 봅시다.

```
infoh@DESKTOP MINGW64 /e/git-rerere (main)

$ cat .git/config 설정 확인

[core]

        repositoryformatversion = 0

        filemode = false

        bare = false

        logallrefupdates = true

        symlinks = false

        ignorecase = true

[rerere]

        enabled = true
```

12.7.4 충돌 및 기록

브랜치를 생성하고 동일한 위치의 코드를 수정했습니다.

▼ 그림 12-16 충돌이 예상되는 저장소



생성한 두 브랜치를 병합합니다. 동일한 파일에서 동일한 위치를 수정했기 때문에 충돌이 발생합니다.

```
infoh@DESKTOP MINGW64 /e/git-rerere (main)


$ git merge feature 병합


Auto-merging hello.htm


CONFLICT (content): Merge conflict in hello.htm


Recorded preimage for 'hello.htm' 충돌


Automatic merge failed; fix conflicts and then commit the result.
```

12.7.5 자동 해결

이전의 충돌 해결 유형은 rerere 기능을 사용하여 기록했습니다. 방금 발생한 문제가 동일한 유형으로 나타난다면, 깃은 기존에 기록한 rerere 정보를 참조하여 코드를 자동으로 수정할 것입니다.

충돌을 해결한 로그 기록을 확인합니다.

```
infoh@DESKTOP MINGW64 /e/git-rerere (main)


$ git log --graph 커밋 기록


*   commit eb528657153e566fd126d8ed2c02c82d47685093 (HEAD → main)


|\  Merge: 240deb0 9a5faec


| | Author: hojin <infohojin@gmail.com>


| | Date:   Wed Feb 20 16:16:35 2019 +0900


| |     solve comflict


| |

...
```

rerere 기능을 실행하려고 이전의 병합 커밋을 강제로 삭제합니다.

infoh@DESKTOP MINGW64 /e/git-rerere (main)

$ git reset --hard HEAD^ 커밋 삭제

HEAD is now at 9b9ac61 world→study

12.8 정리

일반적인 사용자는 깃의 고급 기능까지 알 필요가 없습니다. 하지만 자신이 프로젝트에서 중요한 자리를 맡았거나 저장소를 운영한다면 좀 더 많은 기능을 알고 있는 것이 좋습니다. 깃의 고급 기능을 이용하면 협업할 때 발생하는 저장소 문제들을 복구하고 해결할 수 있습니다.

1 자신의 깃허브 계정에서 만든 저장소 주소를 사용하세요.